EC

Generative Model:

- A generative model ing model that learns patterns from existin new data.
- It focuses on underst rated rather than only predicting labels.
- After learning from an produce new samples that resemble the ori
- These samples are n ata but are newly created based on learned pat
- Generative models are used to generate images, text, audio, and other forms of data.

How Generative Models Learn:

- Generative models l number of data examples during trai
- They identify comm s, structures, styles, or relationships present
- The model gradually stribution that explains how the data is form
- After training, the m s learned distribution to create new data.
- The generated data a it follows the same statistical patterns as the original dataset.
- This process enables the model to produce new, meaningful examples rather than copying existing ones

- When you teach a child to draw animals, the child does not memorize each picture exactly.
- Instead, the child ob d slowly learns common patterns such as bod eyes, ears, and tails.
- Over time, the child tanding of what generally makes an animal loo
- Later, the child can as never shown before by combining these ningful way.
- A generative model the same manner.
- During training, the model is shown many data samples, such as images, text, or audio.

- Using neural networks, it learns the underlying structure and statistical patterns present in th
- Rather than storing t earns a representation of the data distribution.
- After training, the m mples by using this learned distribution, s realistic but is not an exact copy of the tra
- This analogy helps e odels are capable of creating new data.
- Just as a child uses create a new drawing, a deep generative model uses learned patterns and probabilities to generate new data that looks realistic and similar to the training data.

Types of Generative Models:

Probabilistic Model

- Based on statistical encies.

Neural Network–Ba

- Deep learning mod ta patterns.

Other Advanced M

- Modern architectur or energy functions.

Probabilistic Models:

Bayesian Networks

- Graph-based models etween variables.
- Useful for causal rea eling, and decision-making.

Hidden Markov Mo

- Sequence models wi rvable outputs.
- Widely used in spee s prediction, and NLP tasks.

Neural Network–Based Models:

Generative Adversar

- Generator creates sy
- Discriminator classi
- Known for producin , videos, and audio.

Variational Autoenc

- Encoder compresses
- Decoder reconstruct
- Can generate new, smooth variations of learned data.

Applications of Generative Models:

- Generative models are u create new data, improve existing data, and suppo g.
- **Key application areas**
- **Image Generation:** Cr g photos, restoring old images, and translating
- **Text Generation:** Prod tbots, summaries, translations, and conten
- **Audio & Music Gener** ech for assistants and composing new music p
- **Synthetic Data:** Building artificial datasets for privacy, training ML models, and data augmentation.
- **Healthcare & Drug Discovery:** Enhancing medical images and designing new molecules for faster research.

Challenges and Limitations of Generative Models:

- Generative models are po ral important challenges that affect their reliability and
- **High Computational Re** dels (especially GANs and diffusion models) demand times, and large memory resources.
- **Variable Output Quality** rrect, blurry, or unrealistic when data is limited, models are complex.
- **Security Risks:** These mo videos, or text, enabling misinformation, imperson .
- **Ethical and Trust Concerns:** The ability to create highly realistic synthetic content raises issues such as deepfakes, privacy violations, and potential misuse.
- **Data Dependence and Bias:** Output quality strongly depends on training data. If data is biased or incomplete, the model will reproduce those biases in its generation.

Discriminative model:

- A discriminative mo earning or deep learning model that focuses o different classes or categories in a datas
- Its primary goal is to correct label to an input based on observed f
- Instead of modeling d, the model directly learns the relationsh the output.
- Discriminative mod e conditional probability of the output given the input.
- Through training on labeled data, the model learns decision boundaries that separate one class from another.

Cont..

- These boundaries he ich class an input sample belongs to.
- Because of this appr els are highly effective for tasks such as cla recognition.
- In simple terms, a di s what makes one class different from anoth
- For example, in spa del is trained on emails labeled as spam or n
- It learns patterns suc ses, or structures that help distinguish spam from legitimate emails.
- When a new email arrives, the model uses the learned decision boundaries to classify it as spam or not spam.

How discriminative models work:

- Discriminative mod onditional probability $P(Y \mid X)$,which rep an output label Y given an input X.
- Their main objective els to inputs rather than to understand how t .
- Unlike generative m dels do not attempt to model the full data d
- Instead, they concen ces between classes.
- First, the model processes the input data and extracts important features that are useful for distinguishing between categories.

- These features may be automatically learned by the model or provided as input, depending on the algorithm. .
- Next, during trainin lationship between these features and the corr
- By observing many sts its parameters to form decision boundaries om another.
- These boundaries re ture space where data points are classified
- Finally, during predi is given, the model analyzes its features side of the learned decision boundary t
- Based on this position, the model assigns the most likely label to the input.
- This process makes discriminative models highly effective for tasks such as classification and recognition.

Examples of Discriminative Models:

- Several common mac the discriminative approach.
- Logistic Regression is ssification problems and learns a decision boun sses based on input features.
- Support Vector Machi parating hyperplane by maximizing the margi elps improve classification robustness.
- Neural Networks are t map inputs directly to output labels by learni tionships.
- Convolutional Neural Networks, a specialized type of neural network, are mainly used for image-related tasks and learn hierarchical visual features to accurately classify images.

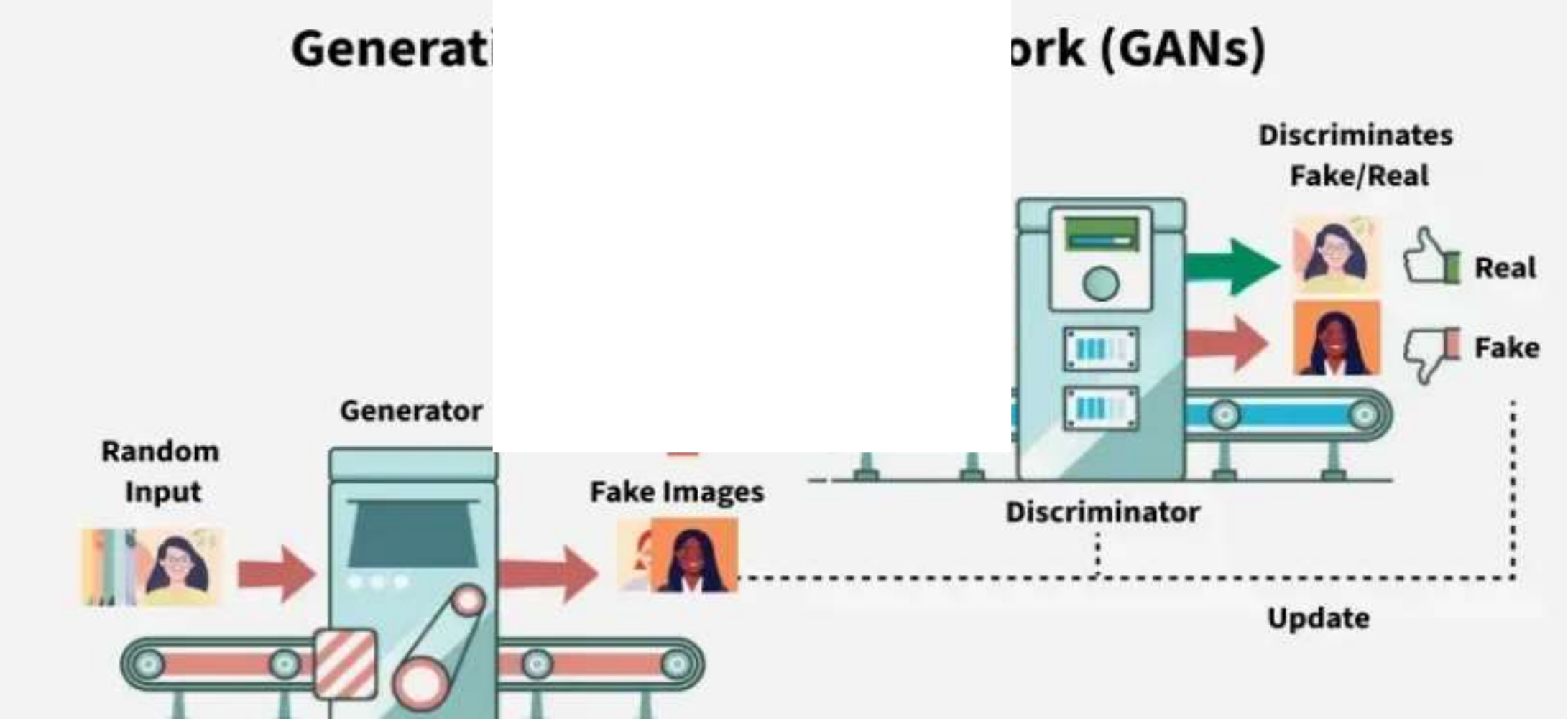
Advantages and Limitations of Discriminati

- One major advantage their efficiency, as they focus only on learning r than modeling the full data distribution.
- They often achieve hi n tasks, especially when trained on large amou
- Discriminative model d optimize compared to many generative mod
- However, a key limita erate new data since they do not model the unde
- Another limitation is their dependence on labeled datasets, which may be costly or difficult to obtain.
- As a result, their performance strongly depends on the quality and quantity of labeled training data.

Generative Adversarial Network (GAN):

- Generative Adversar lp machines to create new, realistic data b xamples.
- It is introduced by Ia am in 2014 and they have transformed how co , videos, music and more.
- Unlike traditional m or classify data, they take a creative way w content that closely resembles real-worl
- This ability helped various fields such as art, gaming, healthcare and data science

Architecture of GAN:



Architecture of GAN:

- GAN consist of t ork together to create realistic synthetic
- They are:
  1. Generator Mode
  2. Discriminator M

1. Generator Model:

- The generator is a d kes random noise as input and generates ges, text, or audio.
- Its goal is to learn th the real data so that the generated samples l
- During training, the rnal parameters using backpropagation bas discriminator.
- The generator does it learns indirectly by trying to fool the dis
- As training progresses, the generator improves its ability to produce realistic samples.

Generator Loss Function:

- The objective of the generator is measured using the following loss function:

$$))$$

- This loss function meas is at fooling the discriminator.
- Here, m is the number represents random noise given as input to the generator
- The term $G(z_i)$denotes ed from the noise.
- The value $D\big(G(z_i)\big)$is t e discriminator that the generated sample is real
- The generator aims to m g the value of $D\big(G(z_i)\big)$.
- In simple terms, the generator wants the discriminator to believe that the fake data it produces is real, meaning the predicted probability should be close to 1.
- When the generator successfully minimizes this loss, it produces more realistic samples that are harder for the discriminator to distinguish from real data.

Generator – Working:

- The generator is a n new data (images, text, audio).
- It starts from rando sforms it into meaningful, realistic
- During training, it le ctured through feedback.
- The following are th ator:
  1. Input: Latent Vect
  2. Fully Connected (
  3. LayersReshape / Projection to Feature Maps
  4. Upsampling / Transposed Convolution Layers
  5. Output Layer (Final Data Generation)
  6. Learning Through Feedback (Training Phase)

Step 1 – Input (Latent Vector / Noise):

- The generator begin alled the latent vector (z).
- This vector is sampl tion, usually Gaussian (Normal) or Unifor
- Example:

  z = [0.2, -0.5, 1 ions)
- Each value in z cont the output, such as:
  - shape
  - color
  - style
  - texture

Step 2 – Fully Connected (Dense) Layers:

- After the latent vector t passes through several fully connected layers
- In these layers, the val combined and transformed through many connect
- Every number in the l y neurons in the next layer, allowing the generato different ways.
- As training progresses e connections so that certain combinations e useful internal patterns.
- Activation functions such as ReLU or Leaky ReLU are applied after each layer to introduce non-linearity.
- This helps the generator learn complex relationships rather than simple straight-line patterns.

Cont..

- Although this stage e part of the final image, it plays an important
- These dense layers t ctor into a richer and more informative int
- This new representa structure that the generator will later e t.
- The output of this st al feature representation that contains more d formation than the original random vector, preparing it for the next stage where it will be reshaped into spatial form.

Step 3 – Reshape / Projection:

- After the fully connecte higher-dimensional feature vector, the generator co small spatial structure.
- This step creates the firs ut.
- For example, if the dens like 8192 values, it is reshaped into somethin ap.
- This means the generato many channels, where each channel holds different
- This spatial structure do , but it introduces the idea of width, height, and depth
- It gives the generator a starting layout that it can gradually expand into a full image in later steps.
- The main purpose of this step is to move from raw numerical features to an organized spatial format so that the generator can begin building shapes, edges, and patterns as it upsamples the feature map.

Step 4 – Upsampling / Transposed Convolutions

- After the small feature increases its size step by step until it becomes a full i
- Each upsampling stage also adds more detail.
- The feature map grows o 8×8, then 8×8 to 16×16, 16×16 to 32×32, and fi
- At each step, transpose followed by convolution are used to enlarge the ima
- In the early expansions ructure, such as the rough position of major shape
- In the middle layers, it starts forming clearer parts such as edges and noticeable patterns.
- In the final layers, it adds fine textures and small details that make the output look realistic.

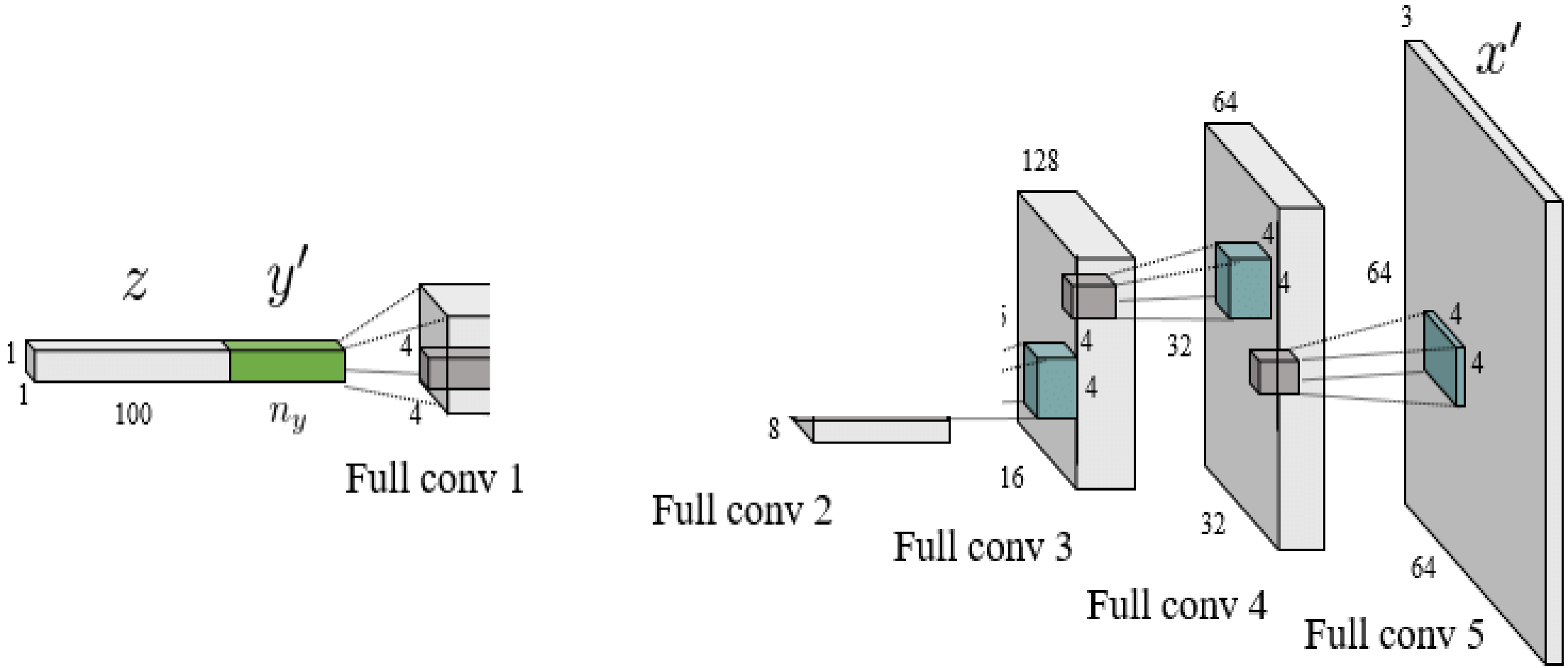
Step 5 – Output Layer:

- In the final stage, the e feature map into the actual output format.
- For images, the last la structure where each value represents a pixel in t
- For text, it produces a h possible token, and for audio, it generates a w
- This layer uses an acti re the values are in the correct range.
- Tanh is used for imag etween −1 and 1.
- Softmax is used for text so each set of values becomes a valid probability distribution.
- The output of this step is the final structured data that looks realistic because the earlier layers have already formed shapes, details, and patterns.

Step 6 – Learning During Training:

- At the beginning of t oduces outputs that look completely random any patterns.
- As training continue g feedback about how close its output is to
- In GANs, the discri back by telling the generator how “fake
- These errors are sent etwork using backpropagation.
- The generator updat ese errors, gradually learning which changes make the output look more realistic.
- Over time, the generator begins to produce outputs with correct structure, believable patterns, and natural variations that resemble the real data it was trained on.

Generator Architecture :



2. Discriminator Model:

- The discriminator ac elps in distinguishing between real and ge
- It learns to improve rough training, refining its parameters to det curately.
- When dealing with i ator uses convolutional layers or other relev elp to extract features and enhance the model’s

Discriminator Loss Function:

- The discriminator loss ell the discriminator can distinguish between r rated by the generator.
- The discriminator is tr ability to real samples and a low probability to fa s, its loss function is defined as:

$$J_D = -\frac{1}{m} \sum_{i=}^{m} \left(1 - D\big(G(z_i)\big)\right)$$

- In this equation, m re ples in a batch.
- The term x_i denotes a real data sample taken from the training dataset, while $G(z_i)$represents a fake data sample generated by the generator from random noise z_i.

- The value $D(x_i)$is the probability predicted by the discriminator that the real sample $x_i$is actually real.
- Similarly, $D\big(G(z_i)\big)$ d by the discriminator that a generated sam
- For real data, the dis be close to 1, which increases log $D\ (x_i)$
- For fake data, it wan 0, which increases log(1−D(G(zi))). .
- By minimizing this l inator learns to correctly classify real samples mples as fake.
- A lower discriminat ification performance.
- This loss function plays a crucial role in the adversarial training process of GANs.

Working of Discriminator:

- Its main role is to di ake data generated by the generator.
- By evaluating the au ovides feedback to the generator, helping it utputs over time.
- **Steps of Discrimina**
  1. Input: Real and G
  2. Preprocessing / N
  3. Convolutional / F
  4. Downsampling / P s
  5. Flatten / Fully Connected Layers
  6. Output Layer (Real vs Fake Score)
  7. Training with Labels (Loss Computation and Backpropagation)
  8. Interaction with Generator (Providing Feedback)

Step 1 – Input (Real and Fake Data):

- During training, the o types of input separately:
  - Real data from the s “real” (1)
  - Fake data generate as “fake” (0)
- For images, each inp 64×3 (height × width × channels).
- For text, inputs are s dings representing words or characters.
- The discriminator pr endently, evaluating whether it is real or fake.
- It does not compare real and fake images side by side; instead, it learns to detect patterns that distinguish real data from generated data across all inputs.

Step 2 – Preprocessing / Normalization:

- Before feeding data puts are preprocessed and normalized.
- For images, pixel va to a standard range (e.g., −1 to 1) or 0 to 1 to le and faster.
- For text or other stru or token encodings convert the input int the network can process.
- This step ensures th tent format and scale, which helps the disc ake patterns effectively.

Step 3 – Convolutional / Feature Extraction L

- The discriminator use ther feature extraction layers for non-image the input.
- Early layers detect lo es, textures, and small shapes.
- Middle layers identify ject parts, repeated patterns, and local structures.
- Deep layers capture gl such as overall structure, symmetry, and realisti .
- Through training, the network automatically learns which features distinguish real data from generated fake data, improving its ability to classify inputs correctly.

Step 4 – Downsampling / Pooling / Strided Convolutions

- After extracting featur ally reduces the spatial dimensions of the inp convolutions.
- This step compresses etwork to focus on larger patterns and global str
- It helps the discrimina rall layout, shapes, and arrangements are reali
- By combining inform s, the network can evaluate both local details and
- Downsampling ensures that deeper layers can make decisions based on the entire input, not just small patches.

Step 5 – Flatten / Fully Connected Layers:

- After convolution and inator produces feature maps that contain lear t.
- These feature maps ha els dimensions (for example, 8×8×256).
- To feed this into fully eature map is flattened into a single 1D vector.
  - Example: 8×8×256 →
- This vector contains a n about the input, combining low-level, res.
- Fully connected layers then process this vector to combine all features and prepare for the final decision (real or fake).

Step 6 – Output Layer (Real vs Fake Score):

- The final layer of th a single score or probability that indi t is real.
- A sigmoid activation the output to a value between 0 and 1.
  - Close to 1 → the in
  - Close to 0 → the in
- This score is then co el (1 for real images, 0 for generated image inator loss.

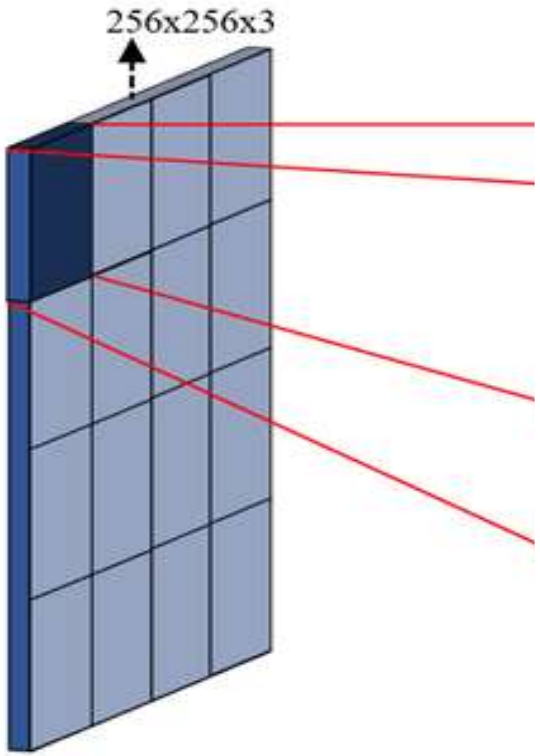
Step 7 – Training with Labels:

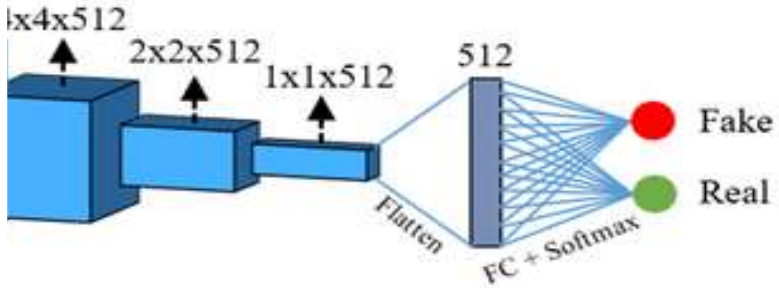
- During training, the to learn:
  - Real images are lab
  - Fake images generat led 0
- For each input, the d ore (real vs fake).
- The loss function m ction is from the true label:
  - Predicting a real im
  - Predicting a fake im
- Gradients of this los ugh the network, updating all weights.
- Over time, the discriminator becomes more accurate at detecting subtle differences between real and generated data.

Step 8 – Interaction with Generator (Providing F

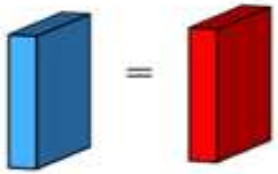
- The discriminator not onl ut also guides the generator during training:
- The generator produces fa scriminator.
- The discriminator evaluat ore indicating how real they look.
- This score is used to com nerator is rewarded when the discriminator is “fooled”
- Backpropagation updates n this feedback.
- Through this adversarial process:
  - The generator improves its ability to create realistic outputs.
  - The discriminator becomes better at spotting fake vs real.
- This competition drives both networks to improve simultaneously, leading to highly realistic generated data.

Discriminator Architecture:





= Temporal Representation Data



= ENCODE

MinMax Loss in Generative Adversarial Networks:

- In Generative Adversa ormulated as a MinMax optimization problem rks: the generator and the discriminator.
- The discriminator trie correctly classify real and fake data, while the g his objective by fooling the discriminator.
- This opposing objecti x loss function:

$$\min_G \max_D V(G,D) = \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

- Here, G is the generator network and D is the discriminator network.

- The distribution $p_{data}(x)$ represents the true data distribution, and $p_z(z)$ represents the s input to the generator.
- The term $D(x)$ is th that a real sample is real, while $D\big(G(z)\big)$ is its sample is real.
- The discriminator ai ctive by assigning high probabilities to real s to generated data.
- In contrast, the gene e same objective by making the discrimi erated data is real.
- This adversarial Mi il the generator learns to produce realistic samples and the discriminator can no longer reliably distinguish between real and fake data.

Summary-How Does a GAN Work:

- A Generative Adver training two neural networks together: t criminator.
- These two networks and improve over time through this competi
- The generator tries t ples, while the discriminator tries to er a given sample is real or generated.
- This adversarial pro learn effectively.

Role of the Generator:

- The generator starts tor, which contains random numbers.
- Using this noise as i ces a fake data sample such as an image or
- Initially, these gener d easily detected by the discriminator.
- Through training, th s from the feedback it receives and gradual c outputs.
- Its main objective is to fool the discriminator into classifying fake data as real.

Role of the Discriminator:

- The discriminator re ples from the training dataset and fake dat e generator.
- It processes each inp lity value between 0 and 1 indicating how rea
- A value close to 1 m real, while a value close to 0 means it is fake.
- The discriminator le dictions with the true labels.
- Over time, it becomes better at identifying fake samples.

Adversarial Learning:

- During training, bot minator learn from each other’s mistakes.
- If the discriminator ata as real and fake data as fake, it improves accurately.
- However, when the amples that the discriminator incorr e generator is rewarded for this success.
- At the same time, th ed for making an incorrect decision.
- This opposite learning objective creates an adversarial relationship between the two networks.

Generator’s Improvement:

- Every time the discr ted data as real, the generator learns wha
- Using this feedback, parameters to produce even more realistic s on.
- Over many training ually learns the patterns and structure of the r
- As a result, the quali les continuously improves.

Discriminator’s Adaptation:

- The discriminator al throughout training.
- It updates its parame tly classifies fake data as real or real data as fa
- This helps the discri e differences between real and generated sampl
- The constant compe tor and discriminator makes both network rate over time.

Training Progression:

- As training progress ously improves its ability to produce realistic
- At the same time, th stronger but gradually finds it harder to dis generated data.
- When the discrimina y tell real data from fake data, it indicates that well-trained or balanced state.
- At this stage, the ge ucing high-quality and realistic synthetic data.
- These generated samples can then be used for various applications such as image generation, data augmentation

Types of GANs:

- There are several ty arial Networks (GANs), each designed to add or improve performance in data generation ta
- Some commonly us
  - Vanilla GAN
  - Conditional GA
  - Deep Convolutio
  - Laplacian Pyram
  - Super-Resolution GAN (SRGAN)

Vanilla GAN:

- Vanilla GAN is the rm of a Generative Adversarial Networ
- It consists of two ne r and a discriminator, both typically imple perceptrons (MLPs).
- The generator create random noise, while the discriminator tries to m fake data.
- Both networks are tr ng a minimax loss function.
- Although Vanilla GAN introduced the basic GAN concept, it often suffers from unstable training and mode collapse.

Conditional GAN (CGAN):

- Conditional GAN ex y introducing an additional condition
- This condition can b ra information that guides the data generation p
- Both the generator a ive this conditional input along with their usu
- This allows the gene types of data rather than random samples.
- For example, CGAN can generate images of a specific class such as handwritten digits or objects based on given labels.

Deep Convolutional GAN (DCGAN):

- Deep Convolutional GA widely used type of GAN designed specifically for
- Unlike Vanilla GANs, D ural Networks (CNNs), which are very effective at lear ges, shapes, and textures in images.
- In DCGAN, the generato layers to gradually build an image from random nois convolutional layers to extract features and classify ima
- Traditional max-pooling rided convolutions are applied to downsample feature m ble learning.
- Fully connected layers are removed, allowing the network to better preserve spatial relationships in images.
- Because of these architectural improvements, DCGANs produce higher-quality and more realistic images with more stable training compared to basic GANs.

Laplacian Pyramid GAN (LAPGAN):

- Laplacian Pyramid a hierarchical, multi-resolution manner.
- It uses multiple gene corresponding to different image resol
- Image generation sta d progressively adds finer details.
- Conditional GANs a el of the pyramid to refine the image.
- This approach produces highly detailed and realistic images while reducing noise.

Super-Resolution GAN (SRGAN):

- Super-Resolution G e the resolution of low-quality images.
- The generator learns ages to high-resolution outputs.
- SRGAN uses advers eptual loss to enhance visual quality.
- It produces sharper i c textures.
- SRGAN is widely used in applications such as medical imaging, satellite imaging, and photo enhancement.

Loss Functions in GANs:

- Loss functions guide) and Discriminator (D) learn during training.
- They measure how w ing and provide gradients for weight updates.
- The discriminator’s lo y it separates real images from fake images.
- The generator’s loss the discriminator.
- GAN training is a min
  - The discriminator tries to maximize correct classification.
  - The generator tries to minimize the chance of being detected.
- The balance between both losses determines whether training is stable or collapses.

Discriminator Loss (Binary Cross-Entropy Loss):

- The discriminator get training:
  - Real data → label 1
  - Fake data from genera
- The discriminator's o
  - Predict **1** for real sam
  - Predict **0** for fake sam
- Loss formula (BCE):

$$LD = -[lo \qquad e))]$$

  - If D correctly identifies real as real → first term is small.
  - If D correctly identifies fake as fake → second term is small.
  - If D gets confused → loss increases.
- This loss ensures the discriminator becomes better at spotting fake images.

Generator Loss (Non-Saturating BCE Loss):

- The generator never put with real images.
- It learns only throug back.
- Goal of generator:
  - Make fake images l → 1
- Loss formula:

$$LG = - lo$$

- If the discriminator t eal → low loss.
- If the discriminator easily rejects it → high loss.
- This loss pushes the generator to learn meaningful patterns and improve realism.

Original GAN Loss Is Unstable:

- The standard GAN lo
- **Vanishing gradients**:
  If the discriminator be generator receives almost no gradient.
- **Non-convergence**:
  Training can oscillate, nt.
- **Mode collapse**:
  The generator may pr s of images.
- **Sensitive to hyperparameters** such as learning rate, batch size, and architecture.
- These limitations led to improved loss functions used in modern GANs.

Least Squares GAN (LSGAN) Loss:

- LSGAN replaces B rror, which smooths gradients and stabili
- Discriminator loss:

  LD = (D(real)
- Generator loss:

  LG = (D(fake)
- Reduces the proble
- Makes fake and real classes easier to separate.
- Produces more stable training and better image quality.
- Useful in tasks like image-to-image translation.

Wasserstein GAN (WGAN) Loss:

- WGAN uses a critic t s instead of probabilities.
- Critic tries to give hig nd lower scores to fake images.
- Loss: LD = D(real) −).
- Scores change smooth eives useful gradients.
- Training becomes sta adients.
- Loss values directly r
- Mode collapse reduces because critic penalizes lack of diversity.
- Requires weight clipping or gradient penalty for stable critic behavior.

Hinge Loss (Used in StyleGAN & BigGAN):

- Hinge loss trains the d les near the decision boundary.
- Loss: LD = max(0, 1 fake)) and LG = −D(fake).
- Real images should h s should have D(fake) ≤ −1.
- Discriminator ignores ns, preventing overfitting.
- Generator receives str ients.
- Improves sharpness a generated images.
- More stable than origi sed in modern high-resolution GANs.
- Reduces training instability and leads to smoother convergence.

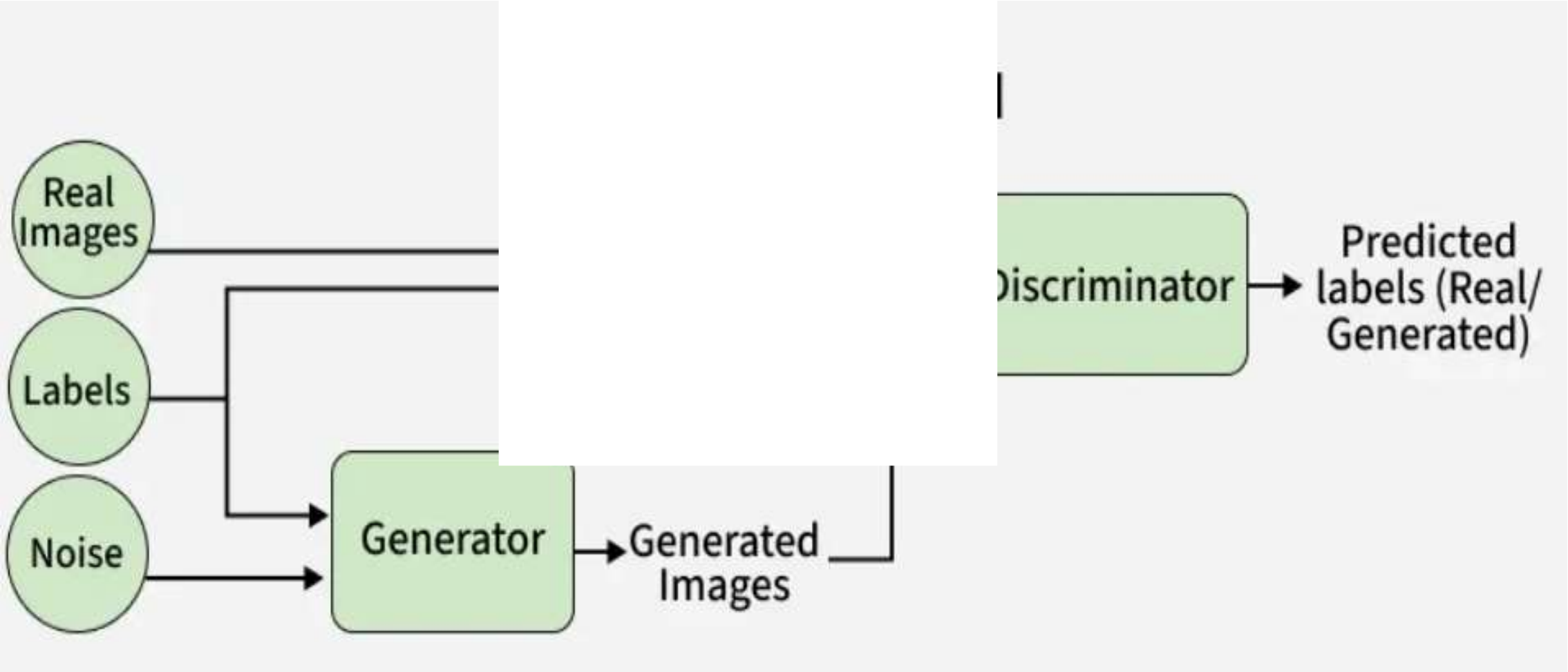
Conditional Generative Adversarial Network (CGAN):

- A Conditional Gener rk (CGAN) is an advanced version of l Network (GAN) that can generate data ba such as a class label, category, or any add
- In a standard GAN, ndom samples without control over the out
- In contrast, a CGAN ation, meaning we can decide what kind of

Example: MNIST Handwritten Digits:

- Consider the MNIS handwritten digits from 0 to 9.
- A normal GAN may it.
- A CGAN can be inst ific digit, such as only digit “7”, by providi ition.
- This makes CGANs d class-specific or targeted data generat

Architecture of Conditional GAN:



Architecture of Conditional GAN:

- A CGAN consists of
  1. Generator (G)
  2. Discriminator (
- The key idea is that ditional conditional information (y).

1. Generator in Conditional GAN (CGAN):

- The generator in a C) is a neural network whose main task is t ks similar to real data, while also matching s a class label.

Inputs to the Genera

- The generator takes

  a) Random Noi

  b) Condition (y

a) Random Noise (z):

- z is a vector of rand pled from a normal or uniform distribution
- Example: z = [0.23,
- Noise ensures:
  - Diversity in generat
  - Different outputs ev
- Without noise, the g he same image every time

b) Condition (y):

- y represents addition sually a class label
- Example (MNIST):

  y = 0, 1, 2, …,
- The condition tells t image to generate

Internal Working of the Generator:

Step 1: Input Pre

1. Noise Vector Crea

- A random noise vect
- Example: 100-dime

2 Condition Encodin

- The condition y (dig a numerical form
- Usually done using
- Example for digit 5:
- y = [0 0 0 0 0 1 0 0 0 0]

Step 2: Combining Noise and Condition:

- The noise vector z a combined
- Common methods:
  - Concatenation
  - Embedding + conca

Example:

- Input to Generator =
- This combined vect
  - Randomness (from z)
  - Control information (from y)

Step 3: Passing Through Fully Connected (Dense) Layer

- After combining the dition label, the resulting input is passed into ected) layer of the generator.
- This dense layer mu rned weights and converts it into a hig vector.
- An activation functi yReLU is then applied to introduce non-linear nt features.
- The output of this si ns condition-specific information learned from the training data.
- This feature vector is then reshaped and further processed to generate the final fake image.

Step 4: Feature Expansion (Image Formation):

- After the dense layer, ed into a small spatial form that can be converted
- This reshaped feature deconvolution (transposed convolution) layers in
- Each deconvolution la mensions of the feature map, gradually expan ge size.
- During this expansion tructure such as strokes and basic shapes.
- As the data passes through successive layers, edges and finer details are refined.
- Finally, the generator produces a complete image of size 28×28 that looks like a realistic handwritten digit.

Step 5: Final Output Formation:

- During the generation formation influences the network at every stag ure, and overall appearance of the image.
- Because the condition rator, features related to the given label are strengt ures are suppressed.
- For example, when th nerator emphasizes curved strokes and patterns ty ids features of other digits.
- In the final stage, the ssed through the output layer.
- This layer uses an activation function such as tanh or sigmoid to scale the output values into a valid pixel range.
- The final result is a 28×28 fake image that closely resembles a handwritten digit and correctly matches the given condition.

What the Generator Learns Over Time:

- During the initial sta ator produces blurry or meaningless images rned useful patterns.
- As training progress rming rough shapes that resemble handwritte
- In the later stages, th me clear and realistic, closely matching rea
- This improvement h ator continuously learns from the feedback p tor.
- Whenever the discri es a fake image, the generator adjusts its weights to reduce errors.
- Over time, this process helps the generator produce high-quality images that can successfully fool the discriminator.

2. Discriminator in Conditional GAN (CGAN):

- The discriminator in as a binary classifier whose main task is t en input image is real or fake while also cons dition.
- It receives two input e conditional information.
- The data sample can ken from the training dataset or a fake ima rator.
- Along with the image, the discriminator also receives the same condition used during generation, such as a class label.

Cont..

- During its operation, ses the image together with the condition to aneously.
- First, it checks whet stic enough to belong to the real dataset.
- Second, it checks w matches the given condition.
- For example, in the ge is provided with the label 3, the discrimi he image resembles a real handwritten digit and whether it truly looks like the digit 3.
- Based on this evaluation, the discriminator outputs a probability indicating whether the input is real or fake.

Internal Step-by-Step Working of the Discriminato

- The working of the n it receives two inputs: an image and its cor el.
- The image can be eit e dataset or a fake sample generated by
- At the same time, th rted into a numerical form, usually throug at it can be processed by the network.
- The condition information is then combined with the image data, either by concatenating it with the input or embedding it into the network layers.

Cont..

- Once the combined i sed through convolutional layers sual features such as edges and shapes fro
- These layers help th d the structure of the image while keeping
- Activation functions plied after each layer to introduce non-linear ning.
- As the data flows through deeper layers, the discriminator learns higher-level features that help distinguish real images from fake ones.

Cont..

- After feature extract d into a single vector and passed through a de
- This dense layer co s and evaluates how realistic the image is the given condition.
- Finally, a sigmoid ac ed in the output layer to produce a probabilit
- A value close to 1 in real and correctly conditioned, while a value close to 0 indicates that the image is fake or does not match the condition.

Adversarial Training Process:

- The generator and dis ltaneously in a competitive manner:

Generator's Objective

- Generate images that l
- Ensure the images ma
- Fool the discriminator es as real

Discriminator's Objec

- Correctly identify real
- Check whether the image matches the condition
- This competition continues until:
  - The generator produces realistic, condition-based images
  - The discriminator can no longer easily distinguish fake images

Loss Function and Training in Conditional GAN (CGAN

- The training of a Conditio n max loss function that creates a competition between the g r.

$$\min_{G} \max_{D} V(D, G) = E_{x\sim \quad \sim p_z(z)} \left[\log\left(1 - D\left(G(z \mid y)\right)\right)\right]$$

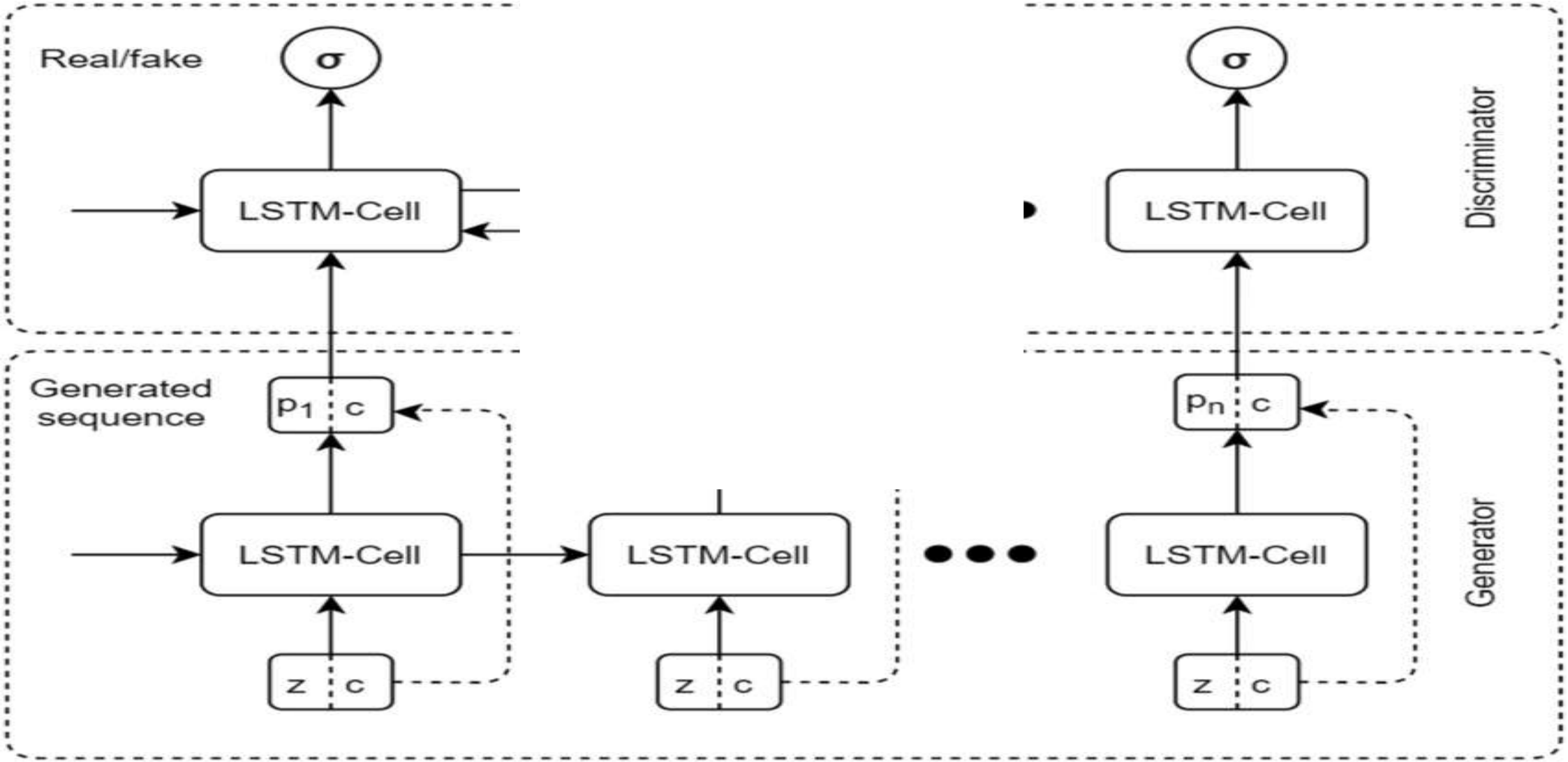
- This loss function is desig to produce realistic data, while the discriminator tries to c e data using the given condition.
- The first part of the loss fu amples.
- It encourages the discriminator to assign a high probability to real images when they are given with the correct condition.
- For example, if a real handwritten digit image is labeled as “3,” the discriminator is rewarded for correctly recognizing it as a real digit 3.

- The second part of the loss function focuses on fake data generated by the generator.
- It pushes the generato discriminator believes are real for the given con
- If the generator produ ,” the loss decreases only when the discriminato at the image is a real digit 5.
- Here, the expected val erformance over many samples.
- The real data distribut data, while the noise distribution refers to r enerator.
- During training, the di bility to distinguish real and fake images, and the generator improves its ability to fool the discriminator.
- This adversarial learning process continues until the generator produces highly realistic images that correctly match the given condition.

Recurrent Conditional GAN (RC-GAN):

- A Recurrent Conditi an extension of the Conditional GAN de nce-based data like text, speech, or video.
- Unlike standard CG le images or data points, RC-GANs produce nto account the temporal or sequential relatio hile conditioning on extra information su

Architecture:



Architecture:

- RC-GAN has two mai riminator, both using recurrent layers (like sequences.
- The **generator** takes:
  - **Random Noise (z** es that introduces diversity.
  - **Condition (y):** La at guides the output (e.g., sentiment, action t
- The **discriminator** ta
  - **Sequence data (x)** dataset or fake sequences from the generator.
  - **Condition (y):** The same label or context to verify consistency.

Generator Working:

- The noise vector z an and embedded into a dense vector to initialize the
- At each time step, the U) generates one element of the sequence (e.g., on rame in a video) while considering its previo
- The hidden state carri ng that the sequence is coherent over time.
- Each output is passed or dense layer to map it to the desired data forma
- This process continue generated, reflecting the condition.
- Example:
  For text generation with condition “positive sentiment,” the generator produces words one by one so that the whole sentence maintains positive sentiment.

Discriminator Working:

- The input sequence bedded and combined.
- The recurrent layers p by step, producing hidden states that su ns.
- At each step or at th network evaluates both realism and conditio
- The final hidden stat se layer with a sigmoid activation to output fake) and 1 (real).

Loss Function and Training:

- RC-GAN uses the sa N:

$$\min_G \max_D V(D,G)$$
$$= E_{x\sim p_{data}(x)}[log \quad g\left(1 - D\left(G(z \mid y)\right)\right)]$$

- The discriminator trie equences as real and fake sequences as fake.
- The generator tries to roducing sequences that are both realistic and consistent with the condition.
- Training alternates between updating the discriminator and generator until the generator produces high-quality sequences.

Applications of RC-GAN:

- Text generation: Sen ces or stories.
- Speech synthesis: G ed on speaker identity or emotion.
- Video generation: G s based on actions.
- Time-series predicti nces conditioned on previous data.

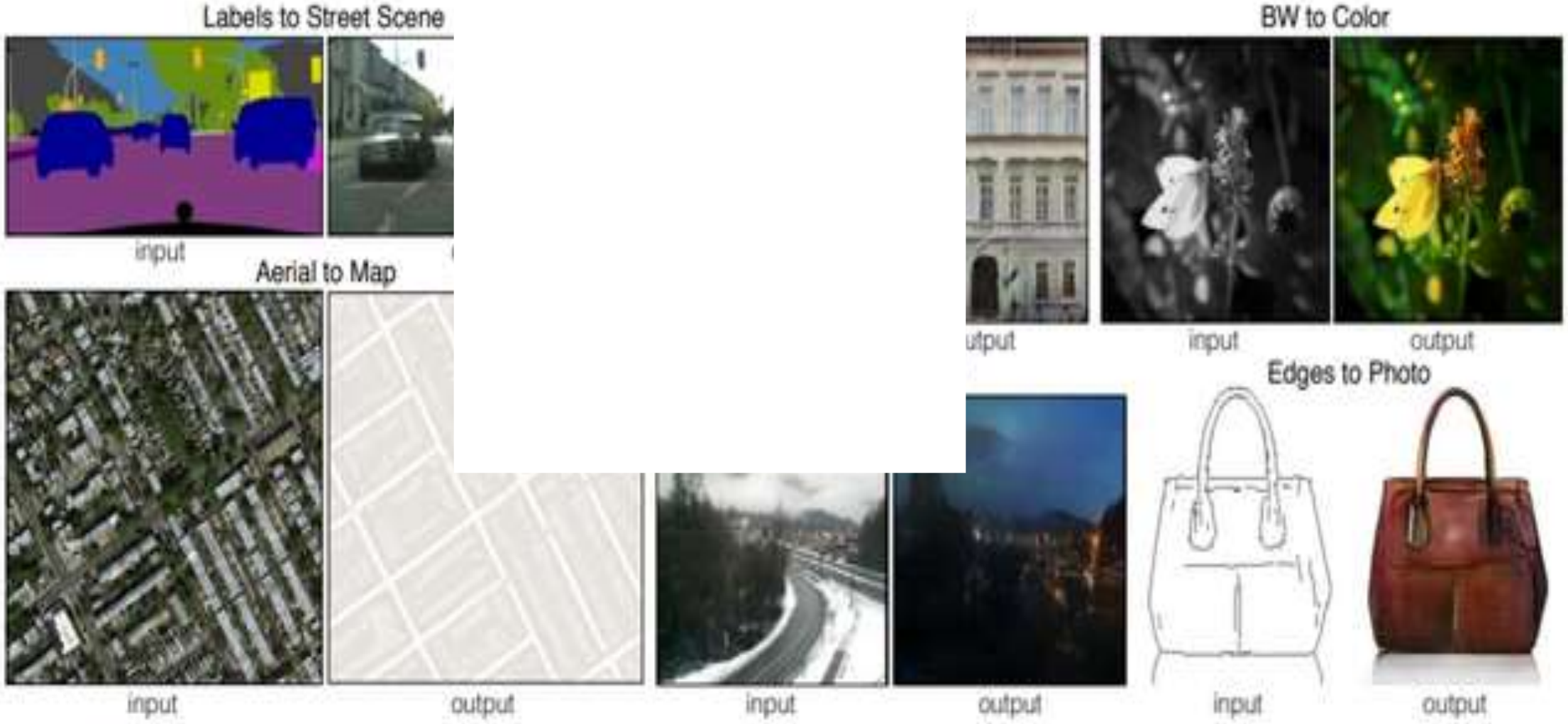
Introduction to Pix2Pix:

- Pix2Pix is a supervi lation technique based on Conditional Generat s (CGANs).
- It was introduced by ley in 2017 to learn a mapping between pa
- The main objective an input image from one domain into a corres another domain.
- Unlike traditional im , Pix2Pix learns this translation automati
- It is widely used in applications such as sketch-to-photo conversion, map-to-satellite image translation, and image colorization.

Image-to-Image Translation:

- Image-to-image tran of converting an image from one visual repr ile preserving its structural informatio
- In this task, the inpu emantically related and usually aligned pixe
- The model learns a f nput image to its corresponding target
- This process requires paired datasets, where each input image has a matching ground-truth output image.
- Pix2Pix is specifically designed to handle such supervised translation problems effectively.

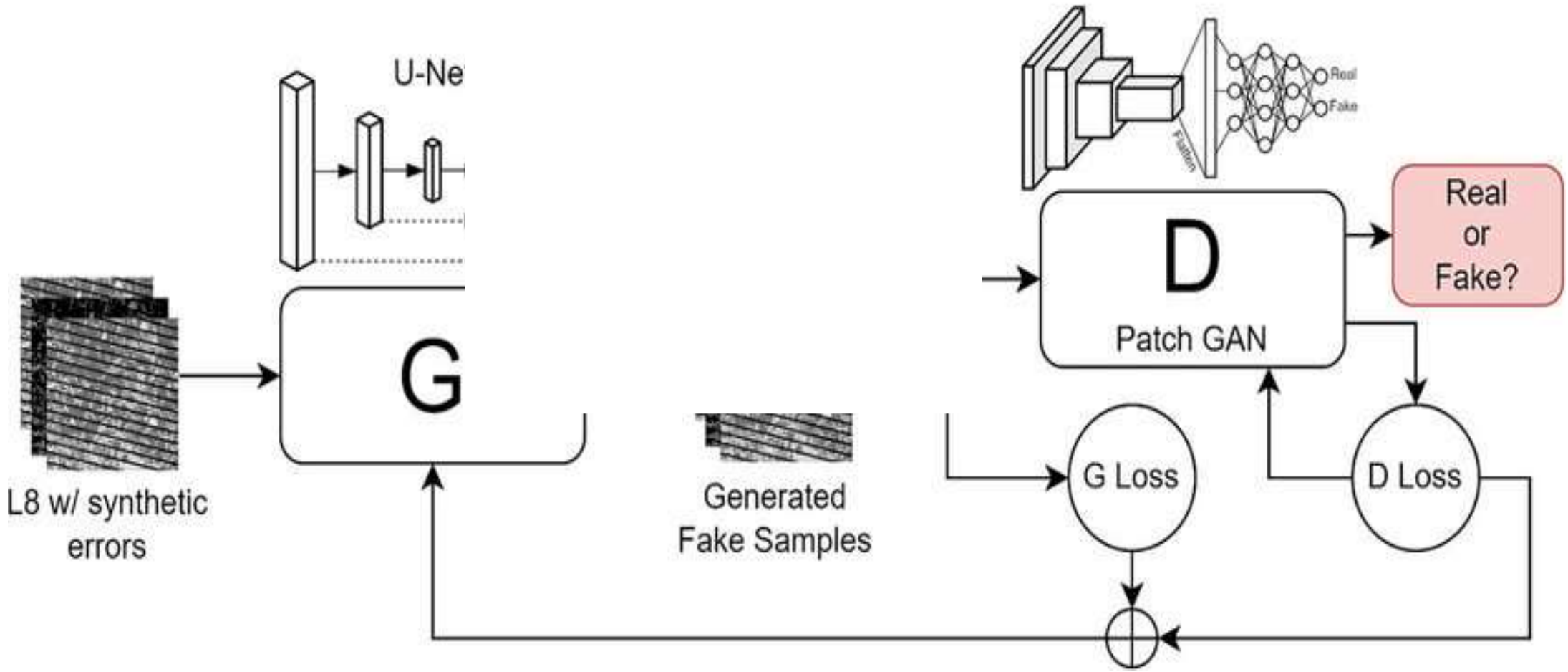
Image-to-Image Translation:



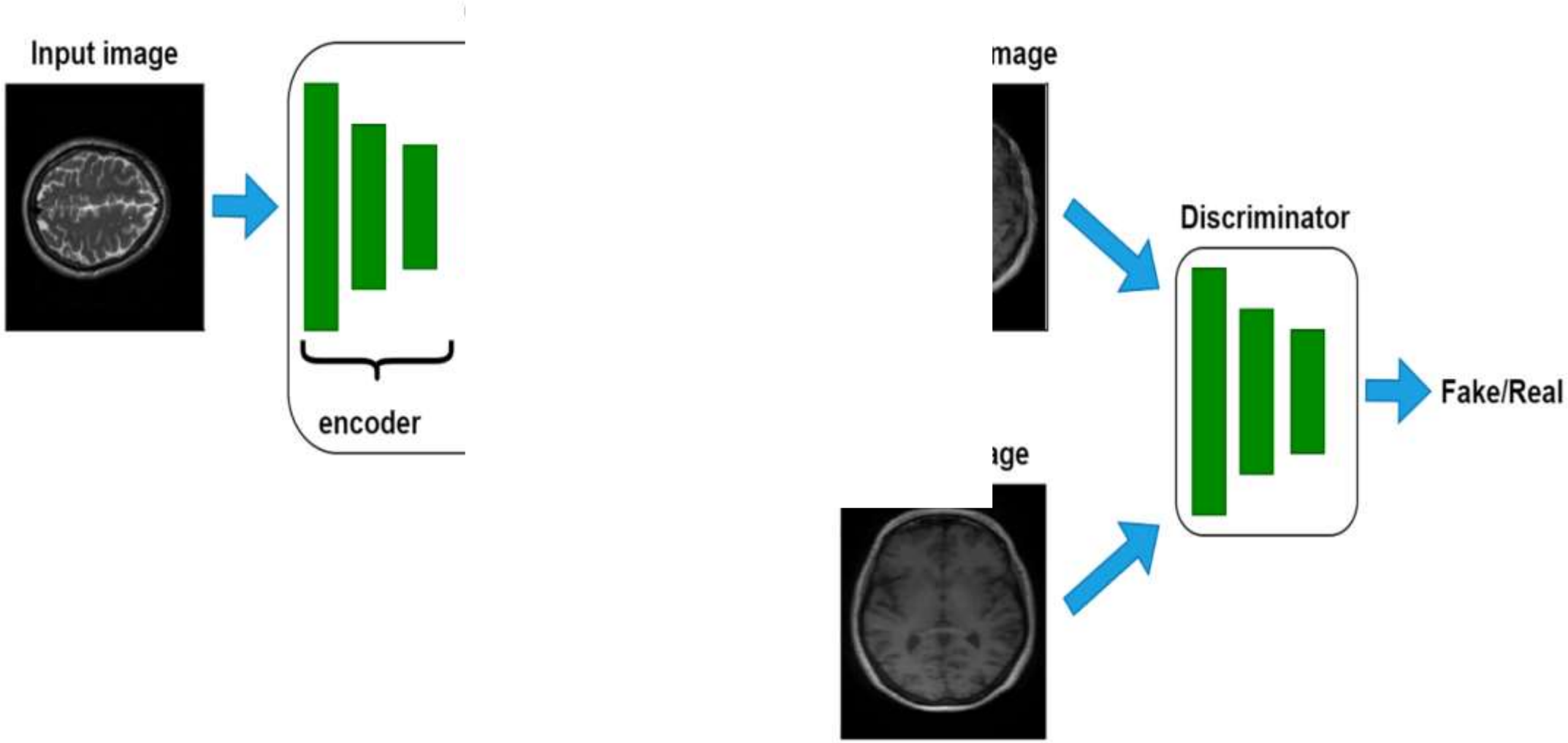
Architecture of Pix2Pix:

- The Pix2Pix archite n components: a generator and a disc
- The generator is res e target image from the input image.
- The discriminator ev rated image is real or fake by comparing it age.
- Both networks are tr an adversarial manner.
- The generator tries to fool the discriminator, while the discriminator tries to correctly identify real and generated images.

Architecture of Pix2Pix:



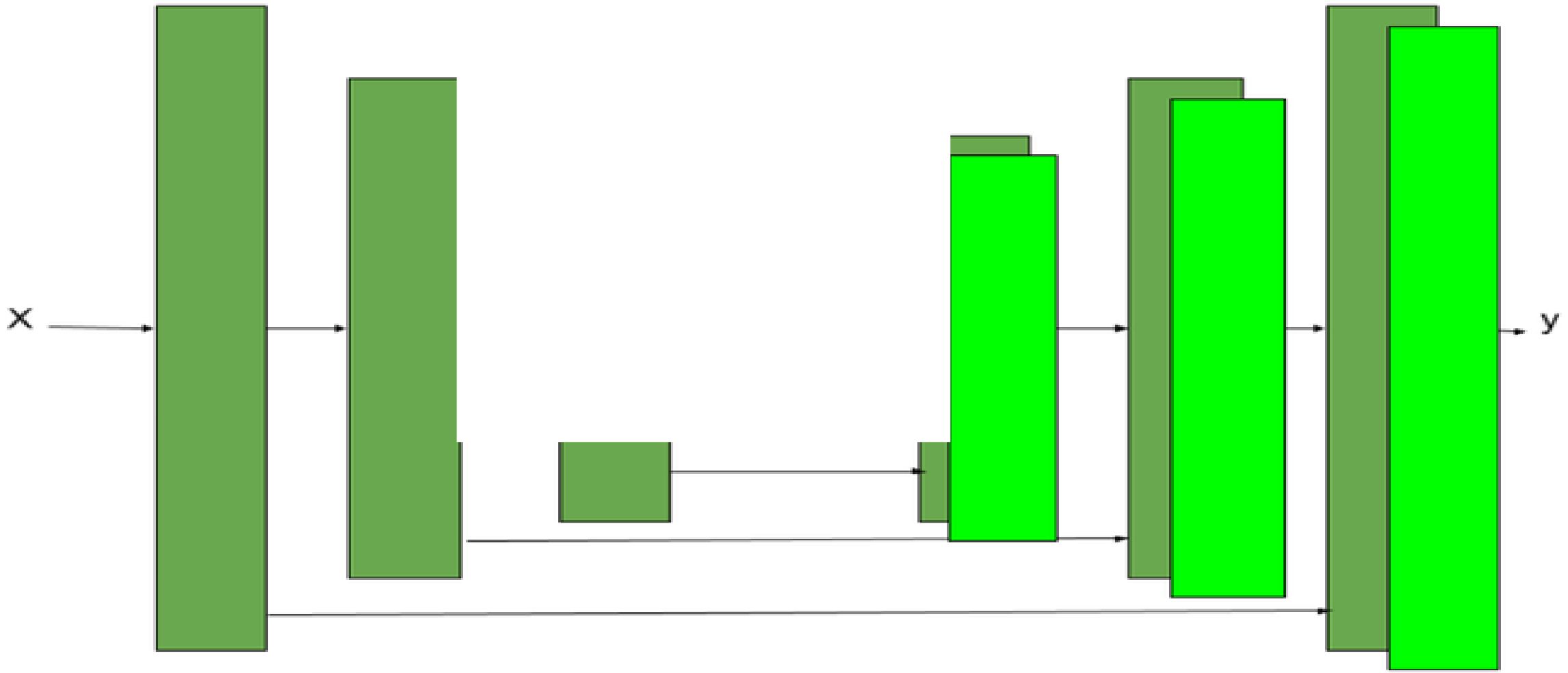
Architecture of Pix2Pix:



Generator Architecture – U-Net:

- The generator in Pix ecture, which is an encoder-decoder net ns.
- The encoder compre a low-dimensional representation by ex es.
- The decoder then re these features to produce the output i
- Skip connections ar nding encoder and decoder layers.
- These connections help retain spatial and low-level information that would otherwise be lost during downsampling.

U-Net architecture:



Encoder Network in Generator:

- The encoder networ s of seven convolutional blocks.
- Each block includes wed by a LeakyReLU activation function
- Batch normalization cept the first one to stabilize training.
- As the image passes spatial resolution decreases while the ls increases.
- The encoder focuses on learning high-level semantic features from the input image.

Decoder Network in Generator:

- The decoder network osed convolutional blocks used for upsa
- Each block contains followed by a convolution layer, ba ReLU activation function.
- The decoder graduall solution of the feature maps.
- Its main purpose is t mage from the encoded representation.
- Skip connections from the encoder provide additional spatial information to improve image quality.

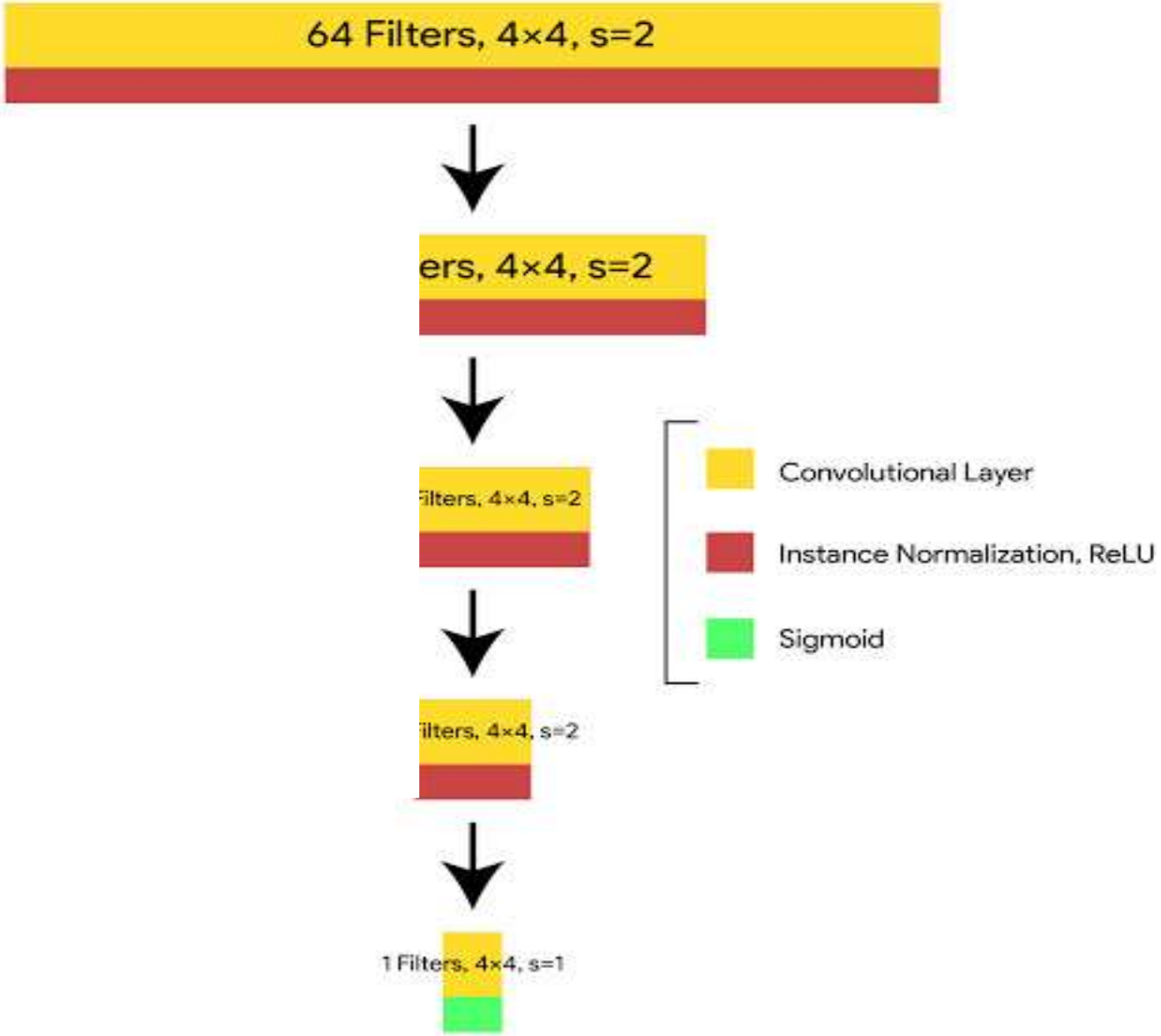
Skip Connections in Pix2Pix:

- Skip connections dir yers to corresponding decoder layers.
- These connections c instead of summing them.
- The encoder capture nt in the image, while the decoder needs infor features are located.
- Skip connections pr ils such as edges and textures.
- As a result, the generated images are sharper and more accurate.

Discriminator Architecture – PatchGAN:

- Pix2Pix uses a Patch of classifying the entire image at once.
- PatchGAN divides th ed-size patches, such as 70×70 pixels, and eva ently.
- Each patch is classifie exture and local patterns.
- Instead of producing a iscriminator produces a grid of real/fake prediction
- The final discriminato raging all patch-level predictions.
- This design allows the discriminator to focus on fine details like edges, textures, and sharp transitions in the image.

PatchGAN:


64 Filters, 4×4, s=2
ers, 4×4, s=2
ilters, 4×4, s=2
ilters, 4×4, s=2
1 Filters, 4×4, s=1
Convolutional Layer
Instance Normalization, ReLU
Sigmoid

Discriminator:

- During training, the irs of images instead of single images.
- The first input pair c t image and the corresponding real t
- This pair represents pping and should be classified as real by
- The second input pa t image along with the image generated by
- This pair represents an incorrect or fake mapping and should be classified as fake.

Cont..

- The discriminator co rated image pairs to learn the difference betwe
- By seeing the same i oth real and fake outputs, the discriminator foc ther than input content.
- This conditional set inator checks whether the generated image the input image.
- Over time, the discri t detecting unrealistic textures and errors.
- This forces the generator to improve its outputs to fool the discriminator.

L1 Loss and L2 Loss in Pix2Pix:

- L1 loss and L2 loss nctions used to measure the difference betwe nd the target image.
- L1 loss calculates th ween corresponding pixels, while L2 loss ifference between pixels. L2 loss heavily pena causes the model to average pixel values ges.
- L1 loss treats all err es edges and fine details better.
- For image-to-image translation tasks, maintaining sharpness and structure is important, so Pix2Pix prefers L1 loss over L2 loss.